

Architecture Overview

Il sistema che ho progettato è una piattaforma di gestione economica personale pensata per essere chiara, estensibile e governabile nel tempo. La complessità del dominio — transazioni, categorie, report, autenticazione, sicurezza, caching, paginazione — richiede un'architettura che separi in modo netto le responsabilità e renda ogni parte del sistema comprensibile e modificabile senza effetti collaterali.

La prima decisione architetturale è stata la **separazione dei livelli**. Ho strutturato il backend in quattro layer distinti:

- **API Layer**, che espone gli endpoint e gestisce la validazione dell'input;
- **Application Layer**, dove risiede la logica dei casi d'uso e l'orchestrazione dei flussi;
- **Domain Layer**, che contiene le entità e le regole del dominio economico;
- **Infrastructure Layer**, che gestisce l'accesso ai dati, il repository pattern, le migrazioni e le integrazioni tecniche.

Questa separazione permette di controllare la complessità: ogni modifica ha un luogo preciso, ogni responsabilità è isolata, ogni componente può evolvere senza rompere gli altri. È la base che consente al sistema di crescere in modo sostenibile.

Ho adottato scelte architetture mirate alla robustezza: autenticazione con JWT access e refresh token, rotazione sicura, autorizzazione basata su ruoli, caching intelligente per categorie e report, logging strutturato con correlation ID, metriche e health check per il monitoring, paginazione su tutte le liste per garantire scalabilità. Ogni decisione è documentata nel **decision log**, dove analizzo alternative, trade-off e motivazioni.

La complessità del sistema non deriva dalla quantità di codice, ma dalla qualità delle interazioni: flussi chiari, dipendenze controllate, confini ben definiti. Il mio obiettivo è sempre lo stesso: costruire software che rimane comprensibile anche dopo mesi, che può essere esteso senza paura, che permette a un team di lavorare con sicurezza e velocità.

Questo overview rappresenta il mio approccio: **architettura prima dell'implementazione**, chiarezza prima della velocità, struttura prima del dettaglio. È così che gestisco la complessità e progetto sistemi che durano nel tempo.

Gestione della complessità e problemi evitati

Sebbene il progetto sia stato costruito da zero, la scelta architetturale non è stata teorica: è nata dalla necessità di evitare problemi concreti che emergono inevitabilmente nei sistemi economici personali, dove transazioni, categorie, report e autenticazione interagiscono tra loro.

In un sistema privo di una struttura chiara, questi domini tendono a intrecciarsi: la logica di calcolo dei report finisce nei controller, le regole del dominio si mescolano con l'accesso ai dati, le entità vengono usate come DTO, e ogni modifica locale ha effetti collaterali globali. Questo porta rapidamente a un codice fragile, difficile da testare e quasi impossibile da evolvere.

La separazione dei livelli ha risolto esattamente questo problema:

- ogni responsabilità ha un luogo preciso;
- i flussi sono chiari e tracciabili;
- il dominio rimane isolato e coerente;
- l'infrastruttura può cambiare senza toccare la logica;
- i controller rimangono sottili e prevedibili;
- i report complessi non contaminano altre parti del sistema.

In altre parole, l'architettura ha evitato la crescita incontrollata della complessità e ha permesso di costruire un sistema che rimane comprensibile, stabile e modificabile nel tempo.

Perché ho scelto questa architettura

Sebbene il progetto sia stato sviluppato come esercizio professionale e non per rispondere a vincoli aziendali, la scelta architetturale non è stata casuale. Ho adottato una struttura a livelli perché è il modello che meglio rappresenta la mia visione di un sistema solido, chiaro e governabile nel tempo.

Nel dominio della gestione economica personale, anche un progetto di dimensioni ridotte può diventare rapidamente complesso: transazioni, categorie, report, autenticazione, sicurezza e caching sono elementi che, se non organizzati correttamente, tendono a intrecciarsi. In architetture meno strutturate, la logica di business finisce nei controller, l'accesso ai dati si mescola con le regole del dominio e ogni modifica locale rischia di generare effetti collaterali globali.

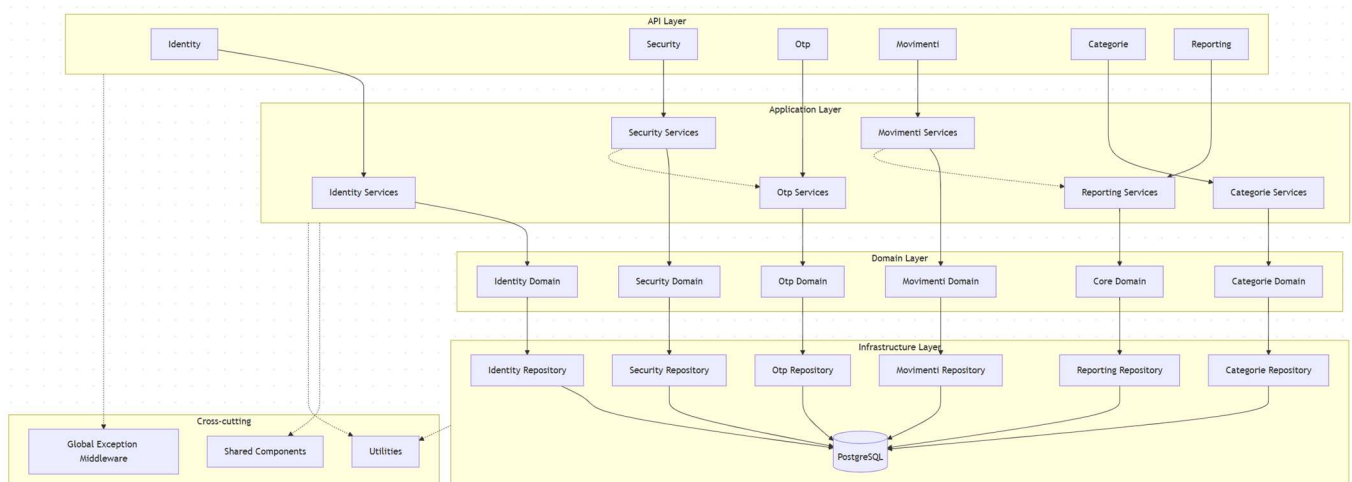
Ho scelto una separazione netta dei livelli proprio per evitare questi problemi. L'architettura a layer permette di:

- isolare le responsabilità;
- mantenere il dominio pulito e coerente;
- rendere l'infrastruttura sostituibile senza impatti sulla logica;
- semplificare l'onboarding e la collaborazione in team;
- facilitare l'evoluzione del sistema nel tempo;
- prevenire la crescita incontrollata della complessità.

Avrei potuto adottare approcci alternativi — architettura modulare, CQRS, domain monolith, microservizi — ma sarebbero stati soluzioni non proporzionate al contesto. La struttura a livelli è la scelta più equilibrata: abbastanza semplice da essere leggibile, abbastanza solida da essere credibile, abbastanza flessibile da poter crescere.

In altre parole, questa architettura non nasce da un vincolo esterno, ma dalla volontà di dimostrare un approccio professionale alla progettazione: costruire sistemi che rimangono chiari, stabili e modificabili nel tempo.

Diagramma Architetturale



Descrizione architetturale

Il backend segue rigorosamente il flusso Controller → Service → Repository → DB, con un layer Domain separato che contiene entità e regole di business indipendenti da DTO e persistenza. I moduli (Identity, Security, Otp, Movimenti,

Categorie, Reporting) sono in gran parte isolati verticalmente: ogni service dipende quasi solo dal proprio repository e dalle proprie entità di dominio, riducendo l'accoppiamento diretto tra moduli.

Le uniche dipendenze esplicite cross-modulo sono: **Security** → **Otp** (AuthService usa OtpService per generare/validare OTP nel flusso di reset password) e **Movimenti** → **Reporting**, mediata da un event bus (Shared.Events): MovimentoService pubblica MovimentoChangedEvent, intercettato da ReportingInvalidationHandler per invalidare la cache dei report, evitando un accoppiamento diretto tra i due moduli.

Il modulo **Shared** fornisce componenti trasversali (middleware di autorizzazione/correlazione, DTO comuni, event bus, health check) usati da tutti gli altri moduli, mentre **utils** offre servizi tecnici condivisi (hashing password, token, sanitizzazione input, gestione eccezioni, connessione DB) usati sia dal layer Application che Infrastructure. Il GlobalExceptionHandler centralizza la gestione degli errori HTTP, eliminando try/catch nei controller.

Rischi architetturali e trade-off

L'architettura a livelli e la separazione verticale dei moduli portano chiarezza e governabilità, ma introducono anche alcuni rischi che ho considerato durante la progettazione.

1. Rischio di over-engineering

Una struttura a layer ben separati può risultare eccessiva per progetti piccoli. Ho mitigato questo rischio mantenendo i moduli verticali indipendenti e limitando il numero di componenti condivisi, evitando di creare un sistema più complesso del necessario.

2. Rischio di proliferazione dei servizi

La divisione netta tra Application e Domain può portare alla creazione di molti service “sottili”, con logica distribuita. Per evitarlo, ho concentrato la logica dei casi d'uso nei service applicativi e mantenuto il dominio pulito ma espressivo, con policy e regole ben definite.

3. Rischio di un modulo Shared troppo centrale

Shared fornisce componenti trasversali, ma se non gestito correttamente può diventare un “God module”. Ho ridotto questo rischio isolando Shared ai soli aspetti tecnici (middleware, event bus, utilities) e mantenendo il dominio completamente indipendente.

4. Rischio di accoppiamento indiretto tramite eventi

L'uso dell'EventBus per collegare Movimenti e Reporting evita dipendenze dirette, ma introduce complessità nella tracciabilità dei flussi. Ho mitigato questo rischio documentando chiaramente gli eventi e mantenendo un numero minimo di interazioni cross-modulo.

5. Rischio di duplicazione tra Domain e Infrastructure

Repository e entità di dominio hanno responsabilità diverse, ma possono sembrare simili. Ho evitato duplicazioni mantenendo il dominio privo di riferimenti alla persistenza e usando DTO specifici per query e repository.

6. Rischio di rigidità nella scalabilità futura

Un'architettura monolitica a moduli verticali è ottima per chiarezza, ma può diventare rigida se il sistema cresce molto. Ho mitigato questo rischio progettando i moduli come “mini-bounded context” indipendenti, così da poterli estrarre in servizi separati senza riscrivere il dominio.

Scalabilità futura ed evoluzione del sistema

L'architettura è stata progettata per essere solida nel presente ma soprattutto per poter crescere in modo controllato. Anche se il sistema nasce come monolite modulare, ogni modulo verticale (Identity, Movimenti, Categorie, Reporting, Security, Otp) è stato strutturato come un mini-bounded context indipendente, con confini chiari e dipendenze minime. Questo approccio permette di scalare il sistema in più direzioni senza richiedere una riscrittura completa.

Scalabilità verticale (aumento del carico) La separazione dei layer consente di ottimizzare singole parti del sistema senza impatti globali:

- caching mirato per report e categorie;
- query ottimizzate nei repository;
- logging e metriche per identificare colli di bottiglia;
- possibilità di introdurre meccanismi di batching o pre-calcolo nei moduli più intensivi.

Scalabilità orizzontale (aumento delle funzionalità) Ogni modulo è isolato e può essere esteso indipendentemente:

- Movimenti può integrare nuove tipologie di transazioni;
- Reporting può aggiungere nuovi report senza toccare il dominio;
- Security può evolvere con MFA, sessioni, audit;
- Identity può integrare ruoli avanzati o permessi granulari.

Scalabilità architetturale (estrazione in servizi) La struttura modulare permette, se necessario, di estrarre singoli moduli in servizi separati:

- Reporting può diventare un servizio autonomo con un proprio database;
- Movimenti può essere isolato come servizio di ledger;
- Identity e Security possono essere consolidati in un servizio Auth centralizzato.

Questa evoluzione è possibile perché i moduli comunicano tramite eventi e componenti condivisi, non tramite dipendenze dirette. L'EventBus riduce l'accoppiamento e rende più semplice spostare parti del sistema senza rompere i flussi.

Scalabilità del team La divisione in moduli verticali facilita la collaborazione: ogni sviluppatore può lavorare su un modulo senza interferire con gli altri, riducendo conflitti, regressioni e complessità cognitiva.

In sintesi, il sistema è progettato per crescere in modo incrementale: prima come monolite modulare, poi come architettura modulare evoluta, e infine — se necessario — come insieme di servizi indipendenti. La scalabilità non è un obiettivo futuro, ma una proprietà intrinseca dell'architettura.

Qualità del codice e manutenibilità

L'architettura definisce la struttura del sistema, ma la qualità del codice è ciò che ne garantisce la longevità. Per mantenere il backend chiaro, coerente e facilmente evolvibile, ho adottato una serie di convenzioni e principi che guidano l'implementazione quotidiana.

Standard di naming e convenzioni Tutti i componenti seguono convenzioni uniformi:

- controller con suffisso *Controller*;
- service applicativi con suffisso *Service* o *UseCase*;
- entità di dominio con nomi espressivi e privi di riferimenti alla persistenza;
- repository con interfacce *I<Nome>Repository* e implementazioni coerenti;
- DTO con nomi che riflettono il contesto (Request/Response). Questa coerenza riduce la complessità cognitiva e rende il codice immediatamente leggibile.

Struttura delle cartelle La struttura del progetto riflette l'architettura: ogni modulo verticale contiene API, Application, Domain e Infrastructure. Questo approccio evita la dispersione dei file e permette di navigare il codice seguendo i confini architetturali, facilitando l'onboarding e la collaborazione in team.

Principi di manutenibilità Il codice segue principi che garantiscono stabilità nel tempo:

- *Single Responsibility*: ogni classe ha un ruolo chiaro.
- *Dependency Inversion*: i service dipendono da interfacce, non da implementazioni.
- *Domain Isolation*: il dominio non conosce DTO, database o infrastruttura.
- *Thin Controllers*: la logica è nei service, non negli endpoint.
- *Explicitness*: nessuna magia, nessun comportamento implicito. Questi principi riducono il rischio di regressioni e rendono il sistema prevedibile.

Test strategy La strategia di test è pensata per coprire i punti critici del sistema:

- test di dominio per verificare regole e policy;
- test dei service applicativi per i casi d'uso principali;
- test di integrazione per repository e query SQL;
- test end-to-end per i flussi più importanti (login, movimenti, report). La separazione dei layer rende i test più semplici: ogni componente può essere isolato e verificato senza dipendenze esterne.

Gestione degli errori e logging strutturato Il *GlobalExceptionHandler* centralizza la gestione degli errori, garantendo risposte coerenti e riducendo duplicazioni nei controller. Il logging strutturato con correlation ID permette di tracciare i flussi end-to-end e identificare rapidamente anomalie o colli di bottiglia.

In sintesi, la qualità del codice non è un aspetto secondario ma una parte integrante dell'architettura: è ciò che permette al sistema di rimanere comprensibile, stabile e modificabile nel tempo, indipendentemente dalla sua crescita.

Impatto sul team e gestione della complessità organizzativa

Sebbene il progetto sia stato sviluppato individualmente, l'architettura è stata progettata pensando a un contesto di team. La struttura a moduli verticali e la separazione dei layer non servono solo a organizzare il codice, ma a rendere il sistema governabile da più persone, con ruoli e competenze diverse.

Onboarding semplificato Ogni modulo (Identity, Movimenti, Categorie, Reporting, Security, Otp) è isolato e contiene API, Application, Domain e Infrastructure. Questo permette a un nuovo sviluppatore di comprendere rapidamente il funzionamento di una parte del sistema senza dover conoscere l'intero backend. La struttura delle cartelle riflette l'architettura, riducendo la complessità cognitiva.

Comunicazione chiara tra sviluppatori La divisione dei moduli e dei layer rende espliciti i confini:

- i controller espongono solo ciò che è necessario;
- i service applicativi definiscono i casi d'uso;
- il dominio contiene regole e policy;
- l'infrastruttura gestisce la persistenza. Questa chiarezza riduce ambiguità e fraintendimenti, facilitando la collaborazione.

Documentazione integrata nell'architettura L'architettura stessa è documentazione:

- naming coerente;
- moduli verticali indipendenti;
- diagrammi dei flussi e dei layer;
- decision log che spiega scelte e trade-off;

- documentazione degli eventi e dei flussi asincroni. Questo permette a un team di capire *perché* il sistema è fatto in un certo modo, non solo *come*.

Divisione del lavoro e ownership dei moduli La struttura modulare permette di assegnare ownership chiara:

- un team può gestire Movimenti;
- un altro Reporting;
- un altro Identity/Security. Ogni modulo è autonomo e può essere sviluppato, testato e rilasciato indipendentemente, riducendo conflitti e regressioni.

Riduzione della complessità organizzativa La separazione dei layer e l'uso dell'EventBus evitano accoppiamenti diretti tra moduli, riducendo la necessità di coordinamento continuo tra team diversi. Le interazioni cross-modulo sono minime e documentate, facilitando la governance del sistema.

In sintesi, anche se oggi non esiste un team, l'architettura è progettata per funzionare bene *con* un team: onboarding rapido, confini chiari, documentazione integrata, ownership definita e riduzione della complessità organizzativa. La qualità di un'architettura non dipende dal numero di persone che la usano, ma dalla sua capacità di scalare quando quelle persone arriveranno.